PC-2025-2026 Course Project: Parallel Time Series SAD Search with OpenMP and CUDA

Yassin Belmaati
yassin.belmaati@edu.unifi.it

Abstract

This report presents the design and implementation of a parallel Sum of Absolute Differences *(SAD) sliding-window search over time series data. Starting from a sequential baseline, two independent parallel implementations are developed: one using OpenMP for shared-memory CPU parallelism and one using CUDA for GPU parallelism. The project explores thread scaling, scheduling strategies, multi-pattern search, shared vs. global GPU memory, and block-size tuning. Both parallel versions are benchmarked against the sequential baseline on synthetic data and on the real-world FordA dataset from the UCR Time Series Archive.*

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

Time series similarity search is a fundamental primitive in data mining, appearing in domains such as financial analysis, IoT sensor monitoring, and signal processing. Given a long time series T of length n and a shorter query pattern P of length m, the *sliding-window SAD search* finds the position i^* that minimises the element-wise ℓ_1 distance between P and the sub-sequence of T starting at i:

$$i^* = \arg\min_{0 \le i \le n-m} \sum_{k=0}^{m-1} |T[i+k] - P[k]|. \quad (1)$$

The naive sequential evaluation of (1) has complexity $O((n-m+1) \cdot m)$, which becomes prohibitive for large n and m. The key observation that makes parallelisation straightforward is that every window evaluation is *independent*: the SAD of window i does not depend on the result of window $j \neq i$. This embarrassingly parallel structure maps naturally both to OpenMP thread-level parallelism and to the SIMT execution model of CUDA.

The remainder of this report is organised as follows. Section 2 describes the shared data structures and infrastructure. Section 3 presents the sequential baseline. Sections 4 and 5 describe the OpenMP and CUDA parallel implementations, respectively. Section 6 reports experimental results.

2. Data Structures and Shared Infrastructure

All three implementations share a common header (`timeseries.h`) and implementation file (`timeseries.cpp`).

Core types. A `TimeSeries` struct wraps a `std::vector<float>` and a string label. A `TimeSeriesDB` holds a collection of series. Search results are returned as a `SearchResult` struct containing the best position, the corresponding SAD value, and the series index (for database searches). Benchmark statistics (mean, standard deviation, min, max, and number of runs) are collected in a `BenchStats` struct.

Benchmark template. A generic `benchmark()` function template accepts any callable match-

ing the signature `SearchResult(const TimeSeries*, const TimeSeries*)` and measures its execution time for a configurable number of repetitions, discarding two initial warm-up calls. This design allows the same infrastructure to be reused for the sequential function, the OpenMP wrapper, and the CUDA lambda without any code duplication.

Data generation and loading. Synthetic data is produced by `ts_generate_sine()` (a parameterised sine wave with optional uniform noise) and `ts_generate_random()` (uniform random floats with a fixed seed). Real data is loaded from the FordA dataset (UCR Time Series Archive via `ts_load_ucr_concat()`, which parses the `.ts` format, strips class labels, and concatenates all series into a single flat vector of the requested length.

3. Sequential Algorithm

The sequential implementation serves as the correctness reference and performance baseline for both parallel versions. The idea is straightforward: slide a window of length m over the time series one position at a time, compute the SAD between that window and the query pattern, and keep track of the position that yields the minimum value. No approximation or pruning is applied; every window is evaluated exactly. Algorithm 1 summarises the procedure.

Algorithm 1 Sequential SAD sliding-window search

Require: time series $T[0..n-1]$, pattern $P[0..m-1]$
Ensure: position i^* minimising SAD

```
best_sad ← +∞,  i* ← 0
for i ← 0 to n − m do
    s ← 0
    for k ← 0 to m − 1 do
        s ← s + |T[i + k] − P[k]|
    end for
    if s < best_sad then
        best_sad ← s,  i* ← i
    end if
end for
return i*
```

3.1. Database search.

`search_db_sequential()` extends the single-series search to a `TimeSeriesDB` by wrapping it in a loop over all series and tracking the global minimum.

4. OpenMP Parallel Implementation

The core idea of the OpenMP implementation is to exploit the independence of window evaluations by distributing the outer sliding-window loop across multiple CPU threads. Each thread processes a disjoint subset of windows, computes its local minimum independently, and contributes to the global result only at the very end through a merge step. This design avoids any synchronisation on the inner loop and keeps contention minimal regardless of the number of threads.

4.1. Single-pattern parallel search

Algorithm 2 shows the parallel search with thread-local reduction.

Algorithm 2 OpenMP parallel SAD search (thread-local reduction)

Require: T, P, number of threads p, schedule σ
Ensure: position i^* minimising SAD

```
best_sad ← +∞,  i* ← 0
set runtime schedule to σ
for thread t ∈ {0, ..., p−1} in parallel do
    local_sad ← +∞,  local_pos ← 0
    for each window i assigned to thread t by schedule σ do
        s ← SAD(T, P, i)
        if s < local_sad then
            local_sad ← s,  local_pos ← i
        end if
    end for
    critical: if local_sad < best_sad then update global best
end for
return i*
```

Each thread maintains its own local best variables throughout the window loop, so no synchronisation is needed on the hot path. A single `critical` section at the end of the parallel region merges the per-thread minima into the global

result; the synchronisation cost is therefore $O(p)$ per call rather than $O(n-m)$.

The schedule kind (`static`, `dynamic`, `guided`, `auto`) and chunk size are runtime parameters, allowing the benchmarks to sweep all combinations without recompilation.

4.2. Multi-pattern parallel search

When multiple query patterns must be searched simultaneously, a different parallelisation axis becomes available: instead of distributing windows within a single search, the patterns themselves are distributed across threads. Each thread receives one or more patterns and performs a complete search for each; all pattern searches proceed concurrently with no shared state between threads. Algorithm 3 illustrates this strategy.

Algorithm 3 OpenMP multi-pattern parallel search

Require: T, patterns $P_1, \ldots, P_k$, threads p
Ensure: $results[j] = \arg\min_i \mathrm{SAD}(T, P_j, i)$

```
for j ∈ {1, ..., k} in parallel (dynamic) do
    results[j] ← SeqSearch(T, P_j)
end for
```

This *task-level* strategy is particularly effective when the number of patterns is large, because it achieves naturally balanced workloads and avoids repeated parallel region setup overhead.

4.3. Database parallel search

For a `TimeSeriesDB` containing multiple series, the loop over series is parallelised with dynamic scheduling. Each thread performs a complete single-series search on its assigned series and merges its local best into the global result through a `critical` section, following the same reduction pattern as the single-pattern case.

4.4. Benchmark suites

The OpenMP binary runs four benchmark suites to characterise different aspects of the parallel implementation:

1. **Thread scaling**: measures mean time and speedup for 1, 2, 4, 8, and 16 threads with static scheduling.
2. **Scheduling strategies**: sweeps all four schedule kinds and several chunk sizes (1, 8, 64, 512) at 8 threads.
3. **Strong scaling**: varies series length from 10 000 to 500 000 and compares sequential vs. 8-thread time at each size.
4. **Multi-pattern**: measures total time for 1, 2, 4, 8, and 16 patterns under the sequential and parallel strategies.

5. CUDA Parallel Implementation

The CUDA implementation exploits the same data-parallel structure as the OpenMP version, but at a much finer granularity: instead of distributing windows across a handful of CPU cores, it launches one GPU thread per window, running up to hundreds of thousands of threads concurrently. Each thread independently computes the SAD of its assigned window and writes the result to a device array; a subsequent reduction step finds the global minimum entirely on the GPU, avoiding any unnecessary data transfer back to the host. Two kernel variants are provided, differing in how the pattern is accessed.

5.1. Baseline kernel (global memory)

Algorithm 4 shows the per-thread computation. Each thread derives its window index from the standard CUDA 1-D grid/block coordinate formula, checks that it is within bounds, and then iterates over all m pattern elements reading from global device memory.

Algorithm 4 CUDA kernel: one thread per window (global memory)

Require: d_T, d_P on device, $n_windows$, m
Ensure: $d_sad[pos] = \mathrm{SAD}(T, P, pos)$

```
pos ← blockIdx.x × blockDim.x + threadIdx.x
if pos ≥ n_windows then
    return
end if
s ← 0
for k ← 0 to m − 1 do
    s ← s + |d_T[pos + k] − d_P[k]|
end for
d_sad[pos] ← s
```

5.2. Optimised kernel (shared memory)

In the baseline kernel, every thread reads the full pattern of m elements from global memory, so each block issues `blockDim.x` redundant loads of the same data. The shared-memory variant eliminates this redundancy: before the SAD computation, all threads in a block cooperatively load the pattern into a fast per-block shared memory buffer. Each thread loads elements at stride `blockDim.x`, a barrier ensures the load is complete, and the SAD loop then reads from shared memory instead of global memory. Algorithm 5 summarises this variant.

Algorithm 5 CUDA kernel: cooperative pattern load into shared memory

Require: d_T, d_P, shared buffer $s_P[0..m-1]$
for $i \leftarrow threadIdx.x$ **to** $m-1$ **step** $blockDim.x$ **do**
  $s_P[i] \leftarrow d_P[i]$
end for
syncthreads()
$pos \leftarrow blockIdx.x \times blockDim.x + threadIdx.x$
if $pos \geq n_windows$ **then**
  return
end if
$s \leftarrow 0$
for $k \leftarrow 0$ **to** $m-1$ **do**
  $s \leftarrow s + |d_T[pos+k] - s_P[k]|$
end for
$d_sad[pos] \leftarrow s$

If the pattern is too large to fit within the device shared memory limit, the launcher falls back to the global-memory kernel automatically.

5.3. Memory management and transfer strategy

GPU buffers for the time series, the pattern, and the output SAD array are allocated once per series/pattern pair and reused across all benchmark runs. The host-to-device data transfer is therefore excluded from the measured time, which covers only the kernel execution and the on-device reduction.

5.4. Result reduction

After the kernel writes one SAD value per window to the device array, the minimum and its position are found using `thrust::min_element()`, which performs the reduction entirely on the GPU. Only the two scalar values (minimum SAD and its index) are transferred back to the host, keeping device-to-host traffic constant and independent of series length.

5.5. Benchmark suites

The CUDA binary runs three benchmark suites:

1. **Block size tuning**: sweeps block sizes 32, 64, 128, 256, and 512 with the global-memory kernel and reports speedup against the sequential baseline.
2. **Global vs. shared memory**: fixes block size to 256, varies pattern length (500, 1 000, 2 500, 5 000), and compares the two kernel variants.
3. **CUDA vs. sequential**: varies series length from 10 000 to 500 000 with the shared-memory kernel at block size 256, reporting speedup at each size.

6. Tests and Results

6.1. Experimental Setup

Hardware.

CPU	`11th Gen Intel(R) Core(TM) i7-11800H`
Physical cores	`8`
Logical threads	`16`
GPU	`NVIDIA T1200 Laptop GPU`
CUDA cores	`1024`
CUDA version	`12.8`
Driver version	`573.71`
VRAM	`4 GB`

Software.
Compiler: `GCC 13.3` NVCC: `12.0`
Flags: `-O3 -march=native -fopenmp -std=c++17`
OS: `Ubuntu 24.04`

Dataset. Two datasets are used. Synthetic data consists of sine waves of configurable frequency

with optional uniform noise, generated deterministically with a fixed seed. Real data is the *FordA* dataset from the UCR Time Series Archive [**?**], which contains 3 601 training series of 500 points each. Series are concatenated in order and truncated at 500 000 points, corresponding to the first 1 000 series.

Methodology. Each configuration is executed $N = 10$ times after two discarded warm-up runs. The reported metric is the mean execution time; summary tables also show the minimum time and speedup relative to the sequential baseline.

6.2. Correctness Validation

Correctness is validated by injecting a known sub-sequence at a fixed position into a random background series and verifying that every implementation returns exactly that position with a SAD below 10^{-4}.

For OpenMP, the test is repeated across all four schedule kinds and four thread counts (1, 2, 4, 8), and separately for the multi-pattern variant with four distinct patterns injected at known offsets. For CUDA, three block sizes and both memory modes are tested, giving six configurations in total. All configurations pass, confirming that neither the parallel reduction nor the GPU computation introduces any correctness issue.

6.3. Results

6.3.1 OpenMP - Thread Scaling

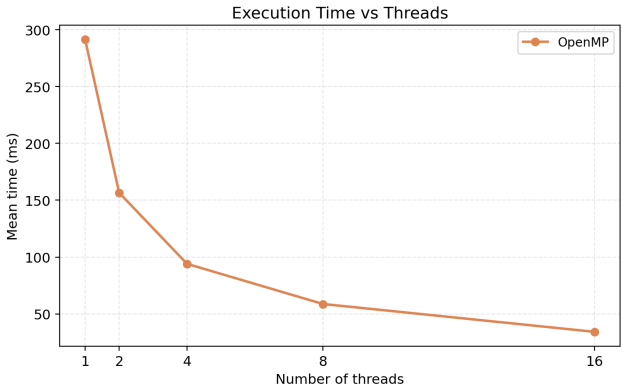

Figure 1. OpenMP execution time vs. number of threads (ts=500 000, pat=5 000, static schedule).

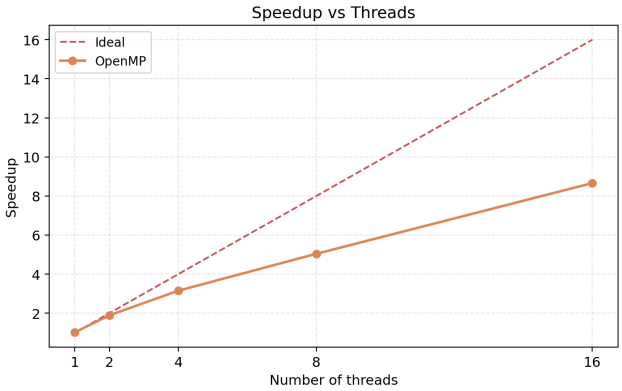

Figure 2. OpenMP speedup vs. ideal linear scaling.

Table 1 reports the measured speedup at each thread count. With 8 physical cores the implementation achieves a speedup of $5.22\times$, corresponding to a parallel efficiency of 65%. Scaling beyond 8 threads still yields a gain, $8.46\times$ at 16 logical threads, but the marginal return drops sharply, as expected when the additional threads are hyperthreads sharing the same physical core resources rather than independent execution units. The gap from ideal scaling (Figures 1 and 2) is consistent with Amdahl's law: the sequential fraction of the workload (memory latency, loop overhead, and the final critical-section merge) limits the achievable speedup regardless of thread count.

Table 1. OpenMP thread scaling summary (ts=500 000, pat=5 000).

Threads	Mean (s)	Min (s)	Speedup
seq	0.2938	0.2898	1.00×
1	0.2916	0.2876	1.01×
2	0.1502	0.1455	1.96×
4	0.0930	0.0781	3.16×
8	0.0563	0.0543	5.22×
16	0.0347	0.0315	8.46×

6.3.2 OpenMP - Scheduling Strategies

As shown in Figure 3, the differences between scheduling strategies are modest: all configurations fall in the range $5.04\times$–$5.94\times$ speedup. The SAD sliding-window workload is highly regular, every window requires exactly m operations, so static scheduling with a moderate chunk size

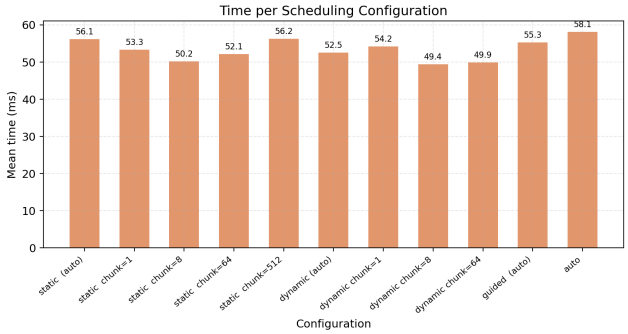

Figure 3. Mean execution time for each scheduling configuration (8 threads, ts=500 000, pat=5 000).

(chunk=8 or chunk=64) performs as well as dynamic, with slightly lower overhead. Dynamic scheduling offers no load-balance advantage here because there is no variability in per-iteration cost.

6.3.3 OpenMP - Strong Scaling and Multi-Pattern

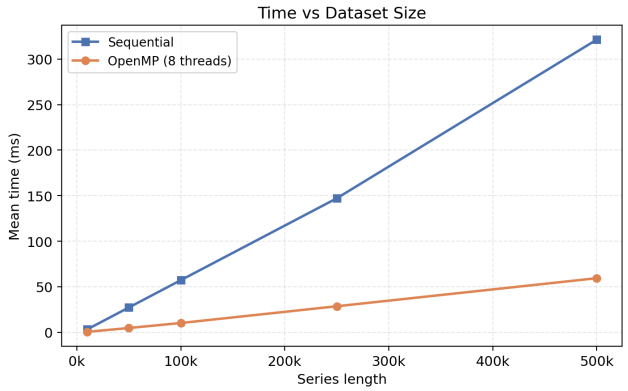

Figure 4. Sequential vs. OpenMP time as series length grows (8 threads, static).

Figure 4 shows that the OpenMP speedup is stable across all tested series lengths ($5.15\times$–$6.93\times$), confirming that the implementation scales with the problem size without additional overheads.

The multi-pattern results (Figure 5) highlight the effectiveness of the task-level parallelisation strategy. With a single pattern the overhead of creating the parallel region yields a negligible gain ($1.06\times$); as the number of patterns grows, each thread receives more independent work and the speedup approaches the thread-scaling ceiling, reaching $5.50\times$ at $k = 8$ and $5.32\times$ at $k = 16$.

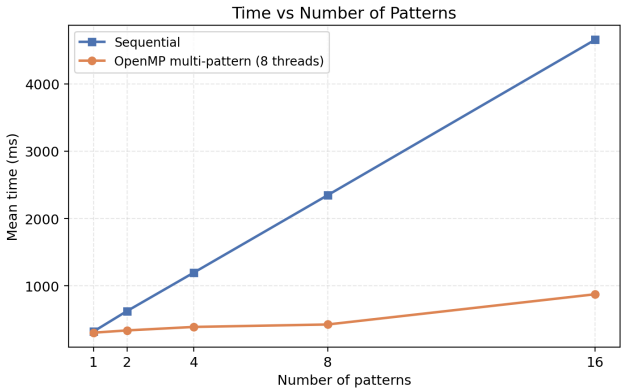

Figure 5. Total search time for k patterns: sequential loop vs. task-parallel OpenMP (8 threads).

6.3.4 CUDA - Block Size Tuning

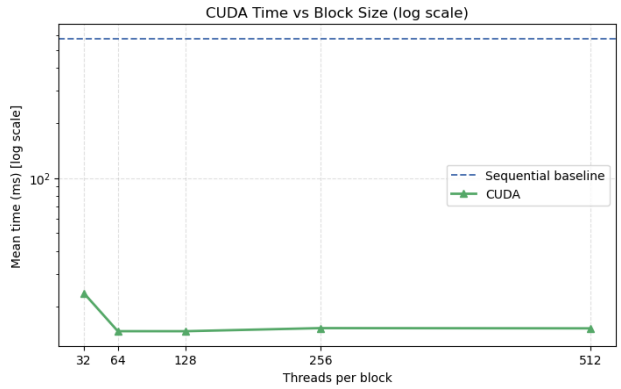

Figure 6. CUDA execution time vs. block size, global-memory kernel (ts=500 000, pat=5 000).

Figure 6 and Table 2 show that block size 32 is significantly slower than all others ($30.94\times$ speedup vs. $\approx$ $38\times$–$48\times$ for larger sizes), because small blocks underutilise the GPU's warp scheduler and reduce memory transaction efficiency.

6.3.5 CUDA - Global vs. Shared Memory

As shown in Figure 7, the shared-memory kernel consistently outperforms the global-memory kernel across all tested pattern lengths, with a ratio ranging from $1.40\times$ at pat=500 to $1.15\times$

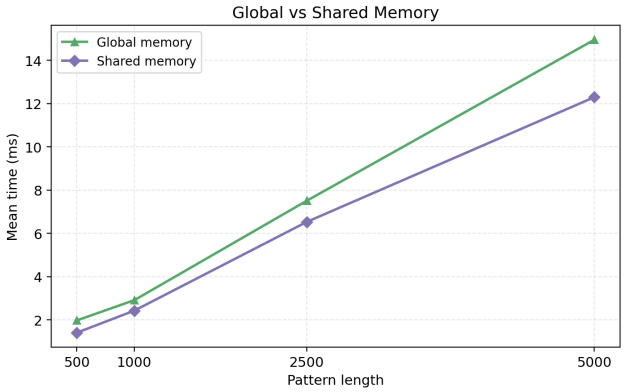

Figure 7. Global vs. shared memory kernel time for increasing pattern length (block=256).

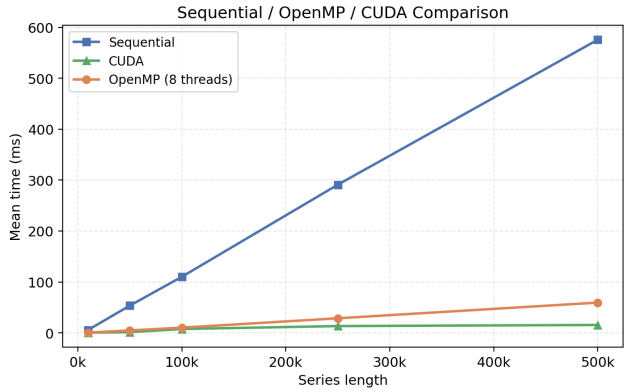

Figure 8. Sequential vs. CUDA execution time as series length grows (block=256, shared-memory kernel).

at pat=2 500. The advantage is largest for short patterns because the pattern-to-window ratio is higher, making the redundant global reads more costly relative to the total work. For longer patterns the inner loop dominates and the relative benefit of caching the pattern diminishes, though an absolute gain remains.

The full summary table (Table 2) confirms that the shared-memory kernel achieves the best speedup across all block sizes tested, peaking at $48.25\times$ with block=512.

Table 2. CUDA summary: mean time and speedup vs. sequential (ts=500 000, pat=5 000).

Config	Mean (ms)	Min (ms)	Speedup
Sequential	576.97	573.48	1.00×
block=32 global	18.65	18.18	30.94×
block=32 shared	42.38	41.92	13.61×
block=64 global	14.98	13.95	38.50×
block=64 shared	20.78	20.10	27.76×
block=128 global	14.94	14.55	38.62×
block=128 shared	13.11	12.66	44.00×
block=256 global	15.03	14.69	38.39×
block=256 shared	12.37	12.04	46.64×
block=512 global	14.86	14.69	38.82×
block=512 shared	11.96	11.73	48.25×

6.3.6 CUDA - Scaling with Series Length

Figure 8 shows that CUDA time grows much more slowly than sequential time as the series length increases, confirming good GPU utilisation at scale. At the largest tested size (500 000 points) the GPU completes the search in 12.4 ms against 577.0 ms for the sequential baseline, a speedup of $46.6\times$. The real-data benchmark on the FordA dataset yields a comparable result ($34.66\times$), confirming that the synthetic benchmarks are representative of real-world performance.

6.4. Conclusions

This project implemented and benchmarked three versions of the SAD sliding-window search: a sequential baseline, an OpenMP CPU-parallel version, and a CUDA GPU-parallel version.

The OpenMP implementation achieves a speedup of $5.22\times$ on 8 physical cores and $8.46\times$ on 16 logical threads, with stable efficiency across series lengths and scheduling strategies. The task-parallel multi-pattern search scales effectively with the number of patterns, reaching $5.50\times$ at $k = 8$.

The CUDA implementation achieves substantially higher speedups, peaking at $48.25\times$ with the shared-memory kernel at block size 512. The shared-memory optimisation provides a consistent advantage over the global-memory baseline, most pronounced for shorter patterns where redundant global reads represent a larger fraction of the total memory traffic.

Both implementations were validated against the sequential baseline across all tested configurations and datasets, confirming correctness in all cases.